

RURALNET TRAINING COLLEGE

Drafted

(will share complete with practical and exercise once done.)

JAVASCRIPT NOTES – PART ONE: THEORY

CHAPTER 1: INTRODUCTION TO JAVASCRIPT

1.1 Definition of JavaScript

JavaScript is a high-level programming language primarily used to make websites interactive and dynamic.

It enables developers to create webpages that respond to user actions, process information, validate forms, perform calculations, display messages and communicate with servers.

JavaScript is one of the three major technologies used in front-end web development:

1. HTML – Provides the structure and content of a webpage.
2. CSS – Controls the appearance and presentation of a webpage.
3. JavaScript – Controls behaviour and interactivity.

1.2 Relationship Between HTML, CSS and JavaScript

HTML, CSS and JavaScript work together to create complete websites.

Technology	Main responsibility	Example
HTML	Structure	Heading, paragraph, image, form
CSS	Presentation	Colours, fonts, spacing, layout
JavaScript	Behaviour	Button clicks, calculations, validation

Analogy:

Think of a website as a human being:

- HTML represents the skeleton.

- CSS represents the appearance and clothing.
- JavaScript represents actions, movement and responses.

1.3 Why JavaScript Is Important

JavaScript is used to:

1. Make websites interactive.
2. Validate information entered into forms.
3. Perform mathematical calculations.
4. Display messages and notifications.
5. Create image sliders and galleries.
6. Control menus and dropdowns.
7. Update webpage content without reloading the entire page.
8. Communicate with web servers.
9. Develop web applications.
10. Create browser-based games.
11. Develop mobile and desktop applications using suitable frameworks and tools.
12. Build server-side applications.

1.4 History of JavaScript

JavaScript was created by Brendan Eich in 1995 while working at Netscape.

It was initially developed to add interactivity to webpages.

Important historical developments:

- 1995 – JavaScript was created.
- 1997 – ECMAScript was standardised.
- 2009 – Node.js was introduced, allowing JavaScript to run outside web browsers.
- 2015 – ECMAScript 2015 (ES6) introduced major language improvements.
- Today – JavaScript is widely used in front-end, back-end and application development.

Important: JavaScript and Java are different programming languages. Their names are similar, but they have different designs, uses and ecosystems.

CHAPTER 2: HOW JAVASCRIPT WORKS

2.1 JavaScript in a Web Browser

When a user opens a webpage, the browser processes its HTML, CSS and JavaScript.

A simplified process is:

1. The browser receives the HTML document.
2. HTML is parsed to create the webpage structure.
3. CSS is processed to determine the presentation.
4. JavaScript is loaded and executed.
5. JavaScript can interact with the webpage and respond to user actions.

2.2 JavaScript Engine

A JavaScript engine is a software component responsible for executing JavaScript code.

Examples include:

- V8 – Used by Google Chrome and Node.js.
- SpiderMonkey – Used by Firefox.
- JavaScriptCore – Used by Safari.

The engine parses and executes JavaScript instructions.

2.3 JavaScript Runtime Environment

A runtime environment provides JavaScript with additional capabilities beyond the language itself.

In a browser, these include:

- DOM APIs.
- Browser events.
- Timers.
- Fetch API.
- Local storage.
- Browser-related objects.

In Node.js, JavaScript can access server-side capabilities through Node.js APIs.

2.4 Client-Side and Server-Side JavaScript

Client-side JavaScript

Runs in the user's browser.

Applications include:

- Form validation.
- Interactive menus.

- Image sliders.
- Updating webpage content.
- Browser-based calculations.

Server-side JavaScript

Runs on a server, commonly using Node.js.

Applications include:

- Processing requests.
- Communicating with databases.
- Authentication systems.
- Building APIs.
- Managing application data.

CHAPTER 3: WAYS OF ADDING JAVASCRIPT TO HTML

JavaScript can be added to an HTML document in three main ways.

3.1 Inline JavaScript

JavaScript is written directly inside an HTML element using an event attribute.

Example:

```
<button onclick="alert('Welcome!')">
    Click Me
</button>
```

The message appears when the user clicks the button.

Advantages:

- Easy to demonstrate simple interactions.
- Suitable for introductory examples.

Disadvantages:

- Mixes HTML and JavaScript.
- Difficult to maintain in large projects.
- Not recommended for complex applications.

3.2 Internal JavaScript

JavaScript is written inside the HTML document using the `<script>` tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
</head>
<body>

    <h1>Welcome</h1>

    <script>
        alert("Welcome to JavaScript!");
    </script>

</body>
</html>
```

The `<script>` tag tells the browser that the enclosed content is JavaScript.

3.3 External JavaScript

JavaScript is written in a separate file with the `.js` extension and linked to the HTML document.

Example project structure:

```
mywebsite/
│
├── index.html
├── style.css
└── script.js
```

HTML:

```
<script src="script.js"></script>
```

JavaScript file:

```
alert("Welcome to my website!");
```

Advantages:

1. Separates HTML and JavaScript.
2. Makes code easier to organise.
3. Allows one JavaScript file to be reused across several webpages.
4. Makes maintenance easier.

3.4 The defer Attribute

The `defer` attribute tells the browser to download an external JavaScript file while parsing HTML, then execute it after HTML parsing is complete.

Example:

```
<head>
    <script src="script.js" defer></script>
</head>
```

This is commonly recommended for scripts that need to access HTML elements.

3.5 The async Attribute

The `async` attribute allows an external JavaScript file to download independently and execute as soon as it is ready.

Example:

```
<script src="script.js" async></script>
```

Unlike `defer`, an async script does not wait for the HTML document to finish parsing. Its execution order is not guaranteed.

Difference:

defer	async
Executes after HTML parsing	Executes as soon as downloading finishes
Preserves order among deferred scripts	Does not guarantee execution order
Suitable for scripts that depend on the parsed HTML	Suitable for independent scripts

---

CHAPTER 4: JAVASCRIPT SYNTAX AND BASIC RULES

4.1 Definition of Syntax

Syntax refers to the rules and structure used when writing instructions in a programming language.

JavaScript has rules that determine how statements, variables, functions and expressions should be written.

4.2 Case Sensitivity

JavaScript is case-sensitive.

This means uppercase and lowercase letters are treated differently.

For example:

```
let name = "Boaz";

let Name = "John";
```

`name` and `Name` are two different variables.

4.3 Statements

A statement is an instruction that tells JavaScript to perform an action.

Example:

```
let age = 25;
```

This statement creates a variable named `age`.

4.4 Semicolons

Semicolons are used to mark the end of JavaScript statements.

Example:

```
let x = 10;
let y = 20;
let total = x + y;
```

JavaScript can automatically insert semicolons in many situations, but using them consistently is a good practice for beginners.

4.5 Whitespace

Whitespace includes spaces, tabs and line breaks.

JavaScript generally ignores unnecessary whitespace.

Example:

```
let name = "Boaz";
```

And:

```
let name="Boaz";
```

Both are valid.

4.6 Comments

Comments are notes written inside code for explanation. They are not executed as JavaScript instructions.

Single-line comment:

```
// This is a single-line comment
let age = 20;
```

Multi-line comment:

```
/*
This is a multi-line comment.
It can contain several lines.
*/
let age = 20;
```

Comments help developers understand and maintain code.

CHAPTER 5: JAVASCRIPT VARIABLES

5.1 Definition of a Variable

A variable is a named storage location used to hold data that a program can access and manipulate.

For example, a student management system may use variables to store:

- Student name.
- Registration number.
- Course.
- Fees paid.
- Balance.

5.2 Declaring Variables

JavaScript provides three main keywords for declaring variables:

1. `var`
2. `let`
3. `const`

A. var

`var` is an older way of declaring variables.

```
var name = "Boaz";
```

A variable declared using `var` can be reassigned and redeclared within the same scope.

B. let

`let` is used when the variable's value may change.

```
let age = 20;

age = 25;
```

The value changes from 20 to 25.

C. const

`const` is used when a variable should not be reassigned.

```
const school = "RuralNet Training College";
```

Trying to reassign this variable causes an error.

5.3 Differences Between var, let and const

Feature	var	let	const
Can be reassigned	Yes	Yes	No
Can be redeclared in the same scope	Yes	No	No
Block-scoped	No	Yes	Yes
Must be initialised during declaration	No	No	Yes

Recommendation: Use `const` by default when reassignment is unnecessary. Use `let` when the value needs to change. Learn `var` because you will encounter it in older code.

5.4 Variable Naming Rules

1. A variable name should begin with a letter, underscore or dollar sign.
2. It must not begin with a number.

3. It must not contain spaces.
4. It must not be a reserved JavaScript keyword.
5. Variable names are case-sensitive.
6. Use meaningful names.

Valid examples:

```
let studentName;
let totalFees;
let _age;
let $price;
let courseCode;
```

Invalid examples:

```
let 2name;
let student name;
let let;
```

5.5 Naming Convention

JavaScript commonly uses camelCase.

Example:

```
let studentName = "Mary";
let totalAmountPaid = 5000;
let registrationNumber = "RN001";
```

CHAPTER 6: JAVASCRIPT DATA TYPES

A data type identifies the kind of value stored in a variable.

JavaScript has several built-in data types.

6.1 String

A string represents text.

Strings can be enclosed in single quotes, double quotes or backticks.

```
let name = "Boaz";
let course = 'Web Development';
let school = `RuralNet`;
```

6.2 Number

The Number type represents numeric values, including whole numbers and decimals.

```
let age = 25;
let fees = 15000;
let price = 250.50;
```

6.3 Boolean

A Boolean represents one of two values:

- `true`
- `false`

Example:

```
let isRegistered = true;
let hasPaidFees = false;
```

6.4 Undefined

A variable that has been declared but has not been assigned a value normally contains `undefined`.

```
let studentName;

console.log(studentName);
```

6.5 Null

`null` represents an intentional absence of a value.

```
let selectedStudent = null;
```

6.6 BigInt

BigInt is used for integers larger than the maximum safe integer supported by the ordinary Number type.

```
let largeNumber = 12345678901234567890n;
```

The `n` at the end indicates a BigInt literal.

6.7 Symbol

A Symbol creates a unique primitive value.

```
let id = Symbol("studentID");
```

Symbols are mainly used in more advanced JavaScript applications.

6.8 Object

An object stores related information using key-value pairs.

```
let student = {
    name: "Mary",
    age: 22,
    course: "ICT Essentials"
};
```

6.9 Array

An array stores multiple values in an ordered collection.

```
let courses = [
    "Web Development",
    "Data Analysis",
    "Graphic Design"
];
```

6.10 Primitive and Non-Primitive Data Types

Primitive types include:

- String.
- Number.
- Boolean.
- Undefined.
- Null.
- BigInt.
- Symbol.

Objects, including arrays and functions, are non-primitive values.

CHAPTER 7: OPERATORS IN JAVASCRIPT

An operator is a symbol or keyword used to perform an operation on values or variables.

7.1 Arithmetic Operators

Used to perform mathematical calculations.

Operator	Meaning	Example

+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
%	Remainder	10 % 3
**	Exponentiation	2 ** 3
++	Increment	x++
--	Decrement	x--

7.2 Assignment Operators

Used to assign or update values.

Operator	Example	Equivalent
=	x = 5	Assign 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5

7.3 Comparison Operators

Used to compare two values.

Operator	Meaning
==	Equal value, with type conversion where applicable
===	Strict equality: equal value and type
!=	Not equal, with type conversion where applicable
!==	Strict inequality
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
let a = 10;
let b = 20;

console.log(a < b);
```

The result is `true`.

Important: Prefer `===` and `!==` when comparing values because they avoid unexpected type conversion.

7.4 Logical Operators

Used to combine or reverse conditions.

Operator	Meaning
`&&`	AND
`` ` ``	
`!`	NOT

Example:

```
let age = 20;
let hasID = true;

console.log(age >= 18 && hasID === true);
```

Both conditions must be true for the complete expression to be true.

7.5 Ternary Operator

The ternary operator provides a short way of writing a conditional expression.

Syntax:

```
condition ? valueIfTrue : valueIfFalse;
```

Example:

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";
```

7.6 Type Operators

The `typeof` operator identifies the type of a value.

```
let name = "Mary";

console.log(typeof name);
```

Result:

`string`

The `instanceof` operator checks whether an object belongs to a particular constructor or class.

CHAPTER 8: CONTROL STRUCTURES

Control structures determine the order in which program instructions are executed.

There are three major categories:

1. Sequential execution.
2. Selection or decision-making.
3. Iteration or repetition.

8.1 Sequential Execution

Instructions are executed one after another in the order they appear.

8.2 Selection Statements

Selection statements allow a program to make decisions.

A. if statement

Executes a block of code when a condition is true.

B. if...else statement

Provides two possible execution paths depending on whether a condition is true or false.

C. else if statement

Allows several conditions to be checked in sequence.

D. switch statement

Selects one block of code from several possible cases.

8.3 Loops

Loops allow a program to repeat instructions.

Common JavaScript loops:

- `for`
- `while`
- `do...while`
- `for...of`
- `for...in`

Loops are useful when processing lists, calculating totals and displaying multiple records.

8.4 Loop Control Statements

- `break` – Terminates a loop or switch statement.
- `continue` – Skips the current loop iteration and proceeds to the next one.

CHAPTER 9: FUNCTIONS

9.1 Definition of a Function

A function is a reusable block of code designed to perform a particular task.

Functions help reduce code repetition and organise programs into manageable sections.

9.2 Benefits of Functions

1. Code reusability.
2. Better organisation.
3. Easier debugging.
4. Reduced repetition.
5. Easier maintenance.

9.3 Parameters and Arguments

A parameter is a variable listed in a function definition.

An argument is the actual value supplied when the function is called.

9.4 Return Values

A function can use `return` to send a result back to the part of the program that called it.

9.5 Types of Functions

Common forms include:

1. Function declarations.
2. Function expressions.
3. Arrow functions.
4. Anonymous functions.
5. Callback functions.
6. Immediately Invoked Function Expressions (IIFEs).

CHAPTER 10: ARRAYS

10.1 Definition

An array is an ordered collection of values stored in a single variable.

Example applications:

- List of students.
- List of courses.
- List of examination marks.
- List of products.

10.2 Array Indexing

Array positions are called indexes.

JavaScript arrays use zero-based indexing.

This means the first element has index `0`, the second has index `1`, and so on.

10.3 Array Properties and Methods

Common array features include:

- `length`
- `push()`
- `pop()`
- `shift()`
- `unshift()`
- `includes()`
- `indexOf()`
- `slice()`
- `splice()`
- `join()`
- `map()`
- `filter()`

- `find()`
- `forEach()`
- `reduce()`

Arrays are useful when a program needs to manage several related values.

CHAPTER 11: OBJECTS

11.1 Definition

An object is a collection of related data and functionality represented using properties and methods.

Example applications:

- Student records.
- Employee information.
- Product details.
- Customer accounts.

11.2 Object Properties

Properties describe an object.

A student object may contain:

- Name.
- Registration number.
- Age.
- Course.
- Fee balance.

11.3 Object Methods

Methods are functions associated with an object.

They describe actions the object can perform.

11.4 Accessing Object Properties

Object properties can commonly be accessed using:

1. Dot notation.

2. Bracket notation.

11.5 The this Keyword

The `this` keyword refers to a value determined by how a function is called.

Inside an ordinary object method, it commonly refers to the object through which the method was called.

Arrow functions behave differently: they do not create their own `this`.

CHAPTER 12: THE DOCUMENT OBJECT MODEL (DOM)

12.1 Definition of DOM

DOM stands for Document Object Model.

It is a programming interface that represents an HTML document as a tree of objects.

JavaScript uses the DOM to access and manipulate webpage elements.

12.2 How the DOM Works

When a browser loads an HTML document, it creates a DOM representation of the document.

For example, an HTML heading becomes an element node in the DOM.

JavaScript can then:

1. Find the heading.
2. Change its text.
3. Change its style.
4. Remove it.
5. Create another element.

12.3 DOM Elements

The DOM represents different parts of an HTML document, including:

- Elements.
- Attributes.
- Text.

- Document structure.

12.4 Common DOM Selection Methods

- `getElementById()`
- `getElementsByClassName()`
- `getElementsByTagName()`
- `querySelector()`
- `querySelectorAll()`

12.5 DOM Manipulation

DOM manipulation refers to changing the structure, content or attributes of a webpage using JavaScript.

Common properties and methods include:

- `textContent`
- `innerHTML`
- `innerText`
- `style`
- `classList`
- `setAttribute()`
- `getAttribute()`
- `createElement()`
- `appendChild()`
- `remove()`

Security note: Inserting untrusted user input through `innerHTML` can create cross-site scripting vulnerabilities. Use `textContent` when inserting plain text.

CHAPTER 13: JAVASCRIPT EVENTS

13.1 Definition of an Event

An event is an action or occurrence recognised by the browser that JavaScript can respond to.

Examples include:

- Clicking a button.
- Typing in an input field.
- Moving the mouse.
- Submitting a form.
- Loading a webpage.

- Pressing a keyboard key.

13.2 Common JavaScript Events

Event	Description
`click`	User clicks an element
`dblclick`	User double-clicks
`mouseover`	Pointer moves over an element
`mouseout`	Pointer moves away
`keydown`	A keyboard key is pressed
`keyup`	A keyboard key is released
`input`	Input value changes
`change`	A form control's value is committed as changed
`submit`	A form is submitted
`load`	A resource finishes loading
`DOMContentLoaded`	HTML parsing is complete

13.3 Event Listeners

An event listener allows JavaScript to respond to a particular event.

The `addEventListener()` method is commonly used to register event handlers.

13.4 Event Object

When an event occurs, the browser can provide an event object containing information about that event.

It can provide details such as:

- The element that triggered the event.
- The event type.
- The mouse position.
- The keyboard key pressed.

13.5 Event Propagation

Event propagation describes how events move through the DOM.

It includes:

1. Capturing phase.
2. Target phase.

3. Bubbling phase.

Understanding event propagation helps developers manage complex interactive webpages.

CHAPTER 14: JAVASCRIPT FORM HANDLING AND VALIDATION

JavaScript can be used to check information entered into HTML forms before submission.

14.1 Form Handling

Form handling involves receiving and responding to information entered by a user.

Examples:

- Student registration.
- Login forms.
- Contact forms.
- Fee payment forms.
- Application forms.

14.2 Form Validation

Form validation checks whether information entered by a user meets specified requirements.

Examples:

1. Checking whether a name is provided.
2. Checking whether an email address is valid.
3. Checking password length.
4. Checking whether two passwords match.
5. Checking whether a number is within an acceptable range.

14.3 Client-Side Validation

Validation performed in the user's browser.

Advantages:

- Immediate feedback.
- Better user experience.
- Reduces unnecessary requests to the server.

14.4 Server-Side Validation

Validation performed on the server after information is submitted.

Server-side validation is essential because browser-side validation can be bypassed.

Important: Never rely on JavaScript alone to protect sensitive information or enforce important security rules.

CHAPTER 15: STRINGS AND STRING METHODS

15.1 Definition

A string is a sequence of characters used to represent text.

Examples include:

- Names.
- Addresses.
- Email addresses.
- Course titles.
- Registration numbers.

15.2 String Methods

JavaScript provides methods for working with text.

Common methods include:

- `length`
- `toUpperCase()`
- `toLowerCase()`
- `trim()`
- `includes()`
- `startsWith()`
- `endsWith()`
- `indexOf()`
- `slice()`
- `replace()`
- `split()`

15.3 Template Literals

Template literals use backticks and allow variables and expressions to be inserted into strings.

They are useful for generating messages and combining text with data.

CHAPTER 16: NUMBERS AND MATHEMATICAL OPERATIONS

JavaScript supports arithmetic operations and provides the built-in `Math` object for mathematical calculations.

16.1 Common Math Methods

Method	Purpose
`Math.round()`	Rounds to the nearest integer
`Math.floor()`	Rounds down
`Math.ceil()`	Rounds up
`Math.abs()`	Returns the absolute value
`Math.max()`	Returns the largest value
`Math.min()`	Returns the smallest value
`Math.sqrt()`	Calculates square root
`Math.pow()`	Raises a number to a power
`Math.random()`	Generates a pseudorandom number

16.2 Number Conversion

JavaScript provides functions for converting values into numbers.

Examples:

- `Number()`
- `parseInt()`
- `parseFloat()`

These are important when processing information received from HTML input fields because input values are generally read as strings.

CHAPTER 17: ERROR HANDLING AND DEBUGGING

17.1 Definition of an Error

An error is a problem that prevents a program from working as intended.

Common categories include:

1. Syntax errors.
2. Runtime errors.
3. Logical errors.

17.2 Syntax Errors

Occur when code violates JavaScript grammar rules.

17.3 Runtime Errors

Occur while a program is executing.

17.4 Logical Errors

Occur when a program runs but produces an incorrect result.

17.5 Error Handling

JavaScript provides:

- `try`
- `catch`
- `finally`
- `throw`

These are used to detect and manage errors.

17.6 Debugging

Debugging is the process of identifying and correcting errors in a program.

Common debugging tools include:

- `console.log()`
- Browser Developer Tools.
- Breakpoints.
- The JavaScript Console.
- The Debugger panel.

CHAPTER 18: JAVASCRIPT ASYNCHRONOUS PROGRAMMING

18.1 Definition

Asynchronous programming allows JavaScript to begin an operation and continue handling other work while waiting for that operation to complete.

This is important for tasks such as:

- Downloading information.
- Communicating with servers.
- Reading files.
- Waiting for timers.
- Retrieving database records through APIs.

18.2 Synchronous Programming

In synchronous programming, instructions generally execute one after another.

A later instruction waits for an earlier instruction to finish.

18.3 Asynchronous Programming

In asynchronous programming, certain operations can be handled without blocking the entire flow of the application.

18.4 Callbacks

A callback is a function passed to another function so that it can be called later.

18.5 Promises

A Promise represents the eventual completion or failure of an asynchronous operation.

A Promise can be:

- Pending.
- Fulfilled.
- Rejected.

18.6 async and await

`async` and `await` provide a more readable way of working with Promises.

They are commonly used when retrieving data from servers.

CHAPTER 19: FETCH API AND COMMUNICATION WITH SERVERS

19.1 Definition

The Fetch API allows JavaScript to make network requests and communicate with servers.

19.2 Common Applications

1. Retrieving student records.
2. Sending login information.
3. Loading course information.
4. Submitting contact forms.
5. Communicating with APIs.
6. Updating website information.

19.3 API

API stands for Application Programming Interface.

An API defines how different software applications or components communicate with one another.

19.4 JSON

JSON stands for JavaScript Object Notation.

It is a text-based format commonly used to exchange data between applications.

JSON data commonly uses:

- Objects.
- Arrays.
- Strings.
- Numbers.
- Booleans.
- Null.

19.5 HTTP Methods

Common HTTP methods include:

- GET – Retrieve information.
- POST – Submit or create information.
- PUT – Replace a resource.
- PATCH – Update part of a resource.
- DELETE – Delete a resource.

CHAPTER 20: WEB STORAGE AND BROWSER STORAGE

JavaScript provides browser APIs that allow websites to store information on a user's device.

20.1 localStorage

`localStorage` stores information in the browser with no automatic expiration date.

Data normally remains available after the browser is closed and reopened, until it is cleared or removed.

20.2 sessionStorage

`sessionStorage` stores information for a particular browsing session.

It is normally cleared when the relevant browser tab or window session ends.

20.3 Cookies

Cookies are small pieces of data stored by the browser and associated with a website.

They are commonly used for:

- Session management.
- User preferences.
- Authentication-related workflows.

Sensitive authentication cookies should be configured securely by the server, using appropriate attributes such as `HttpOnly`, `Secure` and `SameSite`.

20.4 Difference Between Browser Storage Types

Feature	localStorage	sessionStorage	Cookies
Storage location	Browser	Browser	Browser

Typical lifetime	Until removed	Session-based	Depends on expiry settings
Automatically sent with HTTP requests	No	No	Yes, when applicable
Typical use	Preferences	Temporary session data	Session management

Security note: Do not store passwords, secret keys or sensitive authentication tokens in ordinary browser storage without understanding the security risks.

CHAPTER 21: OBJECT-ORIENTED PROGRAMMING IN JAVASCRIPT

21.1 Definition

Object-Oriented Programming (OOP) is a programming approach that organises software around objects containing data and behaviour.

21.2 Main OOP Concepts

1. Classes.
2. Objects.
3. Encapsulation.
4. Inheritance.
5. Polymorphism.
6. Abstraction.

21.3 Classes

A class is a blueprint used to create objects with similar properties and methods.

21.4 Constructors

A constructor is a special method used when creating a new instance of a class.

21.5 Inheritance

Inheritance allows one class to extend another class and reuse or customise its features.

21.6 Encapsulation

Encapsulation involves grouping data and related methods together while controlling access to internal details.

21.7 Polymorphism

Polymorphism allows different objects to respond to the same operation in different ways.

CHAPTER 22: JAVASCRIPT MODULES

Modules allow developers to divide a large JavaScript application into smaller, manageable files.

22.1 Export

The `export` keyword makes functions, variables or classes available to other modules.

22.2 Import

The `import` keyword allows one module to use features exported by another module.

22.3 Benefits of Modules

1. Better code organisation.
2. Reusability.
3. Easier maintenance.
4. Reduced duplication.
5. Easier teamwork.

Modules are commonly used in modern JavaScript applications.

CHAPTER 23: JAVASCRIPT SECURITY

Security is an important part of JavaScript development.

Common Security Risks

A. Cross-Site Scripting (XSS)

Occurs when malicious content is inserted into a webpage and executed in a user's browser.

B. Unsafe Input Handling

Occurs when applications accept user input without proper validation or handling.

C. Exposing Sensitive Information

JavaScript running in a browser is visible to users. Secret keys, database passwords and other confidential information should never be placed in client-side JavaScript.

D. Insecure Authentication

Authentication must be properly handled on the server. Hiding a button or checking a user's role only in JavaScript does not secure an application.

Security Best Practices

1. Validate data on the server.
2. Treat user input as untrusted.
3. Avoid inserting untrusted data using `innerHTML`.
4. Keep confidential credentials on the server.
5. Use HTTPS.
6. Apply appropriate authentication and authorisation controls.
7. Keep software dependencies updated.

CHAPTER 24: JAVASCRIPT LIBRARIES AND FRAMEWORKS

A library is a collection of reusable code that developers can use to perform common tasks.

A framework provides a more structured environment for developing applications.

Popular JavaScript Libraries and Frameworks

1. React

A JavaScript library used for building user interfaces.

Website: https://react.dev/

2. Angular

A web application framework used to build structured applications.

Website: https://angular.dev/

3. Vue.js

A JavaScript framework used for building user interfaces.

Website: https://vuejs.org/

4. jQuery

A JavaScript library that historically simplified DOM manipulation, event handling and AJAX operations.

Website: https://jquery.com/

5. Express

A web framework commonly used with Node.js to build server applications.

Website: https://expressjs.com/

6. Node.js

A JavaScript runtime that allows JavaScript to run outside the browser.

Website: https://nodejs.org/

Advantages of Libraries and Frameworks

1. Faster development.
2. Reusable code.
3. Established development patterns.
4. Access to a large developer community.
5. Support for building complex applications.

Recommendation: Learn core JavaScript before moving to frameworks such as React or Angular.

CHAPTER 25: APPLICATIONS OF JAVASCRIPT

JavaScript is used in many areas of technology.

25.1 Website Development

Used to create interactive websites and web applications.

25.2 Form Validation

Used to check information entered by users.

25.3 Student Management Systems

Used to manage student records, fees, courses and examination information.

25.4 E-commerce Systems

Used to manage shopping carts, product interactions and checkout interfaces.

25.5 Data Visualisation

Used to display charts, graphs and interactive reports.

25.6 Mobile Application Development

JavaScript-based frameworks can be used to develop mobile applications.

25.7 Desktop Application Development

Tools such as Electron allow developers to build desktop applications using web technologies.

25.8 Game Development

JavaScript can be used to create browser-based games and interactive experiences.

25.9 Server-Side Development

Node.js allows JavaScript to be used for back-end application development.

END OF JAVASCRIPT THEORY – PART ONE

GENERAL REVISION QUESTIONS

1. Define JavaScript.
2. Explain the relationship between HTML, CSS and JavaScript.
3. State five applications of JavaScript.
4. Differentiate between client-side and server-side JavaScript.

5. Explain three ways of adding JavaScript to HTML.
6. Differentiate between `var`, `let` and `const`.
7. Define a variable and give three examples of valid variable names.
8. Explain five JavaScript data types.
9. Differentiate between primitive and non-primitive data types.
10. Explain the purpose of arithmetic, assignment and comparison operators.
11. Differentiate between == and ===.
12. What is a control structure?
13. Explain the purpose of loops.
14. Define a function and state four advantages of using functions.
15. Differentiate between an array and an object.
16. What is the DOM?
17. State five common JavaScript events.
18. Differentiate between client-side and server-side form validation.
19. Define debugging and explain three types of programming errors.
20. Differentiate between synchronous and asynchronous programming.
21. What is a Promise?
22. Explain the purpose of the Fetch API.
23. Differentiate between localStorage and sessionStorage.
24. Explain four concepts of Object-Oriented Programming.
25. State five security practices that should be observed when developing JavaScript applications.

PART TWO: PRACTICAL JAVASCRIPT

Course: ICT – Web Development
Level: Beginner to Intermediate
Prerequisite: HTML and CSS fundamentals

CHAPTER 1: SETTING UP A JAVASCRIPT PROJECT

1.1 Creating a Project Folder

Before writing JavaScript, create a project folder.

Example folder structure:

```
JavaScript-Practice/
│
├── index.html
├── style.css
└── script.js
```

Each file has a specific responsibility:

- `index.html` – Contains the webpage structure.
- `style.css` – Controls the appearance.
- `script.js` – Contains JavaScript instructions.

1.2 Creating the HTML File

Open VS Code and create a file named `index.html`.

Write the following code:

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Practice</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <h1>My First JavaScript Project</h1>
    <p>Welcome to JavaScript programming.</p>
```

```
    <script src="script.js"></script>
</body>
</html>
```

1.3 Creating the CSS File

Create `style.css`:

```
body {
    font-family: Arial;
    background-color: #f2f2f2;
    text-align: center;
    padding: 30px;
}

h1 {
    color: darkgreen;
}
```

1.4 Creating the JavaScript File

Create `script.js`:

```
alert("Welcome to JavaScript!");
```

Open `index.html` in a browser.

A message box will appear.

Important: The HTML file connects the three technologies. The CSS file controls appearance, while the JavaScript file controls behaviour.

1.5 Connecting JavaScript to HTML

There are two common ways of connecting an external JavaScript file.

At the bottom of the HTML body:

```
<script src="script.js"></script>
```

Or inside the head using `defer`:

```
<head>
    <script src="script.js" defer></script>
</head>
```

Both approaches allow JavaScript to access the webpage after its HTML has been parsed. For beginners, use one method consistently.

CHAPTER 2: DISPLAYING OUTPUT IN JAVASCRIPT

JavaScript provides different ways of displaying information.

2.1 Using console.log()

`console.log()` displays information in the browser's developer console.

JavaScript:

```
console.log("Hello World!");
console.log(100);
console.log(20 + 30);
```

To view the output:

1. Open the webpage.
2. Press `F12`.
3. Select the Console tab.

Expected output:

```
Hello World!
100
50
```

Application: Useful when testing code and identifying errors.

2.2 Using alert()

`alert()` displays a message in a popup dialogue box.

```
alert("Welcome to RuralNet Training College!");
```

The user must dismiss the message before continuing to interact with the page.

2.3 Using document.write()

`document.write()` writes content directly into the HTML document.

```
document.write("Welcome to JavaScript!");
```

It can also display calculations:

```
document.write(20 + 30);
```

Important: Avoid using `document.write()` in modern applications after a page has loaded because it can replace the entire document. Use DOM manipulation instead.

2.4 Using innerHTML

`innerHTML` allows JavaScript to insert HTML content inside an existing element.

HTML:

```
<p id="message"></p>
```

JavaScript:

```
document.getElementById("message").innerHTML =
    "<b>Welcome to JavaScript!</b>";
```

The paragraph displays the message in bold.

2.5 Using textContent

`textContent` inserts or retrieves plain text from an element.

HTML:

```
<p id="message"></p>
```

JavaScript:

```
document.getElementById("message").textContent =
    "Learning JavaScript is interesting!";
```

Unlike `innerHTML`, `textContent` does not interpret inserted text as HTML.

Comparison of Output Methods

Method	Purpose
`console.log()`	Displays information in the console
`alert()`	Displays a popup message
`document.write()`	Writes into the document
`innerHTML`	Inserts HTML content into an element
`textContent`	Inserts plain text into an element

CHAPTER 3: JAVASCRIPT VARIABLES IN PRACTICE

A variable stores information that can be used by a program.

3.1 Declaring Variables

```
let studentName = "Mary";
let age = 22;
let course = "ICT Essentials";

console.log(studentName);
console.log(age);
console.log(course);
```

3.2 Using var

```
var school = "RuralNet Training College";

console.log(school);
```

3.3 Using let

```
let amount = 5000;

amount = 7000;

console.log(amount);
```

Output:

```
7000
```

3.4 Using const

```
const taxRate = 0.16;

console.log(taxRate);
```

A `const` variable cannot be reassigned.

3.5 Combining Variables

```
let firstName = "Boaz";
let lastName = "Vidal";

let fullName = firstName + " " + lastName;

console.log(fullName);
```

Output:

```
Boaz Vidal
```

3.6 Template Literals

Template literals use backticks.

They allow variables to be inserted into text using `${}`.

```
let name = "Mary";
let course = "Web Development";

let message = `Hello ${name}, welcome to ${course}.`;

console.log(message);
```

Output:

```
Hello Mary, welcome to Web Development.
```

PRACTICAL EXERCISE 1

Create a JavaScript program that stores:

1. Student's full name.
2. Registration number.
3. Course.
4. Age.
5. Total fees.
6. Amount paid.

Display all the information in the browser console using `console.log()`.

CHAPTER 4: DATA TYPES AND TYPE CONVERSION

4.1 Working with Strings

```
let name = "John";
let course = "Computer Studies";

console.log(name);
console.log(course);
```

4.2 Working with Numbers

```
let price = 500;
let quantity = 4;

let total = price * quantity;

console.log(total);
```

4.3 Working with Booleans

```
let isRegistered = true;
let hasPaid = false;

console.log(isRegistered);
console.log(hasPaid);
```

4.4 Checking Data Types

Use `typeof` to determine the type of a value.

```
let name = "Mary";
let age = 23;
let fees = 15000;
let registered = true;

console.log(typeof name);
console.log(typeof age);
console.log(typeof fees);
console.log(typeof registered);
```

4.5 Converting Strings into Numbers

HTML input fields normally provide their values as strings.

Consider:

```
let a = "20";
let b = "30";

console.log(a + b);
```

Output:

```
2030
```

This happens because the + operator joins the two strings.

Convert the values into numbers:

```
let a = Number("20");
let b = Number("30");

console.log(a + b);
```

Output:

```
50
```

Other conversion methods:

```
let age = parseInt("25");
let price = parseFloat("150.50");
```

Important: Check whether the conversion succeeded when processing user input. Invalid text can produce `NaN` (Not a Number).

CHAPTER 5: JAVASCRIPT OPERATORS IN PRACTICE

5.1 Arithmetic Operators

```
let a = 20;
let b = 10;

console.log(a + b); // Addition
console.log(a - b); // Subtraction
console.log(a * b); // Multiplication
console.log(a / b); // Division
console.log(a % b); // Remainder
console.log(a ** b); // Exponentiation
```

5.2 Assignment Operators

```
let total = 100;

total += 50;
```

```
console.log(total);

total -= 20;
console.log(total);

total *= 2;
console.log(total);

total /= 2;
console.log(total);
```

5.3 Comparison Operators

```
let age = 20;

console.log(age == 20);
console.log(age === 20);
console.log(age > 18);
console.log(age < 18);
console.log(age >= 18);
console.log(age !== 25);
```

The results are Boolean values: `true` or `false`.

5.4 Logical Operators

```
let age = 22;
let hasID = true;

console.log(age >= 18 && hasID === true);
```

The `&&` operator means both conditions must be true.

```
let isStudent = true;
let isStaff = false;

console.log(isStudent || isStaff);
```

The `||` operator means at least one condition must be true.

```
let isBlocked = false;

console.log(!isBlocked);
```

The `!` operator reverses a Boolean value.

PRACTICAL EXERCISE 2: FEES CALCULATION

A student is expected to pay KSh 25,000 and has paid KSh 18,000.

Write JavaScript instructions to calculate:

1. Total expected fees.
2. Amount paid.
3. Remaining balance.
4. Percentage of fees paid.

CHAPTER 6: ACCEPTING USER INPUT

JavaScript can receive information from users through popup boxes and HTML input elements.

6.1 Using prompt()

`prompt()` displays a dialogue box that allows a user to enter information.

```
let name = prompt("Enter your name:");

alert("Welcome " + name);
```

6.2 Receiving Numbers Using prompt()

```
let age = Number(prompt("Enter your age:"));

alert("Next year you will be " + (age + 1));
```

Note: `prompt()` returns text or `null` if the user cancels. Convert and validate the input before using it in calculations.

6.3 Using confirm()

`confirm()` displays a dialogue box with OK and Cancel options.

```
let answer = confirm("Do you want to continue?");

console.log(answer);
```

- OK returns `true`.
- Cancel returns `false`.

6.4 Using HTML Input Fields

HTML:

```
<!DOCTYPE html>
<html>
<head>
    <title>User Input</title>
</head>
<body>

    <h2>Enter Your Name</h2>

    <input type="text" id="name" placeholder="Your name">

    <button onclick="greetUser()">Submit</button>

    <p id="result"></p>

    <script src="script.js"></script>

</body>
</html>
```

JavaScript (`script.js`):

```
function greetUser() {

    let name = document.getElementById("name").value;

    document.getElementById("result").textContent =
        "Welcome " + name;

}
```

Explanation:

- `getElementById("name")` locates the input field.
- `.value` retrieves what the user typed.
- `textContent` displays the result.
- `greetUser()` is executed when the button is clicked.

CHAPTER 7: CONDITIONAL STATEMENTS

Conditional statements allow a program to make decisions.

7.1 The if Statement

```
let age = 20;
```

```
if (age >= 18) {
    console.log("You are an adult.");
}
```

The message is displayed only when the condition is true.

7.2 The if...else Statement

```
let age = 16;

if (age >= 18) {
    console.log("You are eligible.");
} else {
    console.log("You are not eligible.");
}
```

7.3 The else if Statement

```
let marks = 75;

if (marks >= 80) {

    console.log("Grade A");

} else if (marks >= 70) {

    console.log("Grade B");

} else if (marks >= 60) {

    console.log("Grade C");

} else if (marks >= 50) {

    console.log("Grade D");

} else {

    console.log("Fail");

}
```

7.4 The switch Statement

The `switch` statement is useful when a variable can have several specific possible values.

```
let day = 3;

switch (day) {

    case 1:
        console.log("Monday");
```

```
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Unknown day");

}
```

7.5 Ternary Operator

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

PRACTICAL EXERCISE 3: STUDENT GRADING SYSTEM

Create a program that accepts a student's marks and assigns a grade.

Use the following grading scale:

Marks	Grade
80–100	A
70–79	B
60–69	C
50–59	D
0–49	E

Requirements:

1. Accept marks from the user.
2. Check that marks are between 0 and 100.
3. Determine the grade.
4. Display the result.
5. Reject invalid entries.

CHAPTER 8: LOOPS

Loops allow JavaScript to repeat instructions.

8.1 The for Loop

The `for` loop is commonly used when the number of repetitions is known.

```
for (let i = 1; i <= 5; i++) {

    console.log(i);

}
```

Output:

```
1
2
3
4
5
```

8.2 The while Loop

A `while` loop repeats as long as its condition remains true.

```
let i = 1;

while (i <= 5) {

    console.log(i);

    i++;

}
```

8.3 The do...while Loop

This loop executes its instructions at least once before checking the condition.

```
let i = 1;

do {

    console.log(i);

    i++;

} while (i <= 5);
```

8.4 The for...of Loop

Used to iterate through values in an iterable object, such as an array.

```
let courses = [
    "ICT Essentials",
    "Web Development",
    "Data Analysis"
];

for (let course of courses) {

    console.log(course);

}
```

8.5 The for...in Loop

Used to iterate over enumerable property keys of an object.

```
let student = {
    name: "Mary",
    age: 22,
    course: "ICT"
};

for (let key in student) {

    console.log(key, student[key]);

}
```

8.6 break and continue

`break` stops a loop.

```
for (let i = 1; i <= 10; i++) {

    if (i === 5) {
        break;
    }

    console.log(i);

}
```

`continue` skips the current iteration.

```
for (let i = 1; i <= 5; i++) {

    if (i === 3) {
```

```
        continue;
    }

    console.log(i);

}
```

PRACTICAL EXERCISE 4

Write a JavaScript program that:

1. Displays numbers from 1 to 100.
2. Displays even numbers between 1 and 50.
3. Calculates the sum of numbers from 1 to 100.
4. Displays the multiplication table of a number entered by the user.
5. Calculates the factorial of a positive integer.

CHAPTER 9: FUNCTIONS IN PRACTICE

A function is a reusable block of code designed to perform a particular task.

9.1 Creating a Function

```
function greet() {

    console.log("Welcome to JavaScript!");

}
```

Calling the function:

```
greet();
```

9.2 Functions with Parameters

```
function greetStudent(name) {

    console.log("Welcome " + name);

}

greetStudent("Mary");
greetStudent("John");
```

9.3 Functions with Multiple Parameters

```
function add(a, b) {
    let total = a + b;
    console.log(total);
}
add(20, 30);
```

9.4 Functions with Return Values

```
function multiply(a, b) {
    return a * b;
}
let answer = multiply(5, 4);
console.log(answer);
```

9.5 Arrow Functions

Arrow functions provide a shorter syntax.

```
const greet = () => {
    console.log("Hello!");
};
greet();
```

An arrow function with parameters:

```
const add = (a, b) => {
    return a + b;
};
console.log(add(10, 20));
```

9.6 Practical Example: Fees Calculator

```
function calculateBalance(totalFees, amountPaid) {
    let balance = totalFees - amountPaid;
    return balance;
```

```
}

let result = calculateBalance(25000, 18000);

console.log("Balance: KSh " + result);
```

9.7 Connecting Functions to HTML Buttons

HTML:

```
<h2>Click the Button</h2>

<button onclick="showMessage()">Click Me</button>

<p id="output"></p>
```

JavaScript:

```
function showMessage() {

    document.getElementById("output").textContent =
        "The button was clicked!";

}
```

CHAPTER 10: ARRAYS IN PRACTICE

An array stores several values in one variable.

10.1 Creating an Array

```
let students = [
    "Mary",
    "John",
    "Peter",
    "Jane"
];
```

10.2 Accessing Array Elements

```
console.log(students[0]);
console.log(students[1]);
console.log(students[2]);
```

Remember that array indexing starts at zero.

10.3 Changing an Array Element

```
students[1] = "David";

console.log(students);
```

10.4 Finding the Array Length

```
console.log(students.length);
```

10.5 Adding Elements

`push()` adds an item to the end.

```
students.push("Grace");
```

`unshift()` adds an item to the beginning.

```
students.unshift("James");
```

10.6 Removing Elements

`pop()` removes the last item.

```
students.pop();
```

`shift()` removes the first item.

```
students.shift();
```

10.7 Searching Arrays

```
let courses = [
    "ICT Essentials",
    "Web Development",
    "Data Analysis"
];

console.log(courses.includes("Data Analysis"));
```

10.8 Looping Through an Array

```
let courses = [
    "ICT Essentials",
    "Web Development",
    "Data Analysis"
];

courses.forEach(function(course) {
```

```
    console.log(course);
});
```

10.9 Using map()

`map()` creates a new array by transforming each element.

```
let numbers = [2, 4, 6, 8];
let doubled = numbers.map(function(number) {
    return number * 2;
});
console.log(doubled);
```

10.10 Using filter()

`filter()` creates a new array containing elements that meet a condition.

```
let marks = [45, 78, 90, 32, 65];
let passed = marks.filter(function(mark) {
    return mark >= 50;
});
console.log(passed);
```

10.11 Using find()

`find()` returns the first element that satisfies a condition.

```
let marks = [45, 78, 90, 32, 65];
let result = marks.find(function(mark) {
    return mark >= 80;
});
console.log(result);
```

CHAPTER 11: OBJECTS IN PRACTICE

Objects store related information using properties and values.

11.1 Creating an Object

```
let student = {

    name: "Mary",
    regNo: "RN001",
    course: "ICT Essentials",
    fees: 25000

};
```

11.2 Accessing Object Properties

Using dot notation:

```
console.log(student.name);
console.log(student.course);
```

Using bracket notation:

```
console.log(student["regNo"]);
```

11.3 Changing Object Properties

```
student.fees = 30000;
```

11.4 Adding New Properties

```
student.phone = "0723000000";
```

11.5 Adding Methods to Objects

```
let student = {

    name: "Mary",
    course: "ICT Essentials",

    greet: function() {

        return "Welcome " + this.name;

    }

};

console.log(student.greet());
```

11.6 Array of Objects

This is particularly useful when managing multiple student records.

```
let students = [

    {
        name: "Mary",
        regNo: "RN001",
        fees: 25000
    },

    {
        name: "John",
        regNo: "RN002",
        fees: 20000
    },

    {
        name: "Peter",
        regNo: "RN003",
        fees: 30000
    }

];

console.log(students[0].name);
console.log(students[1].regNo);
```

CHAPTER 12: THE DOCUMENT OBJECT MODEL (DOM)

The DOM allows JavaScript to access and manipulate HTML elements.

12.1 Selecting an Element by ID

HTML:

```
<h1 id="heading">Welcome</h1>
```

JavaScript:

```
let heading = document.getElementById("heading");

console.log(heading);
```

12.2 Selecting Elements by Class

HTML:

```
<p class="message">First paragraph</p>
<p class="message">Second paragraph</p>
```

JavaScript:

```
let messages = document.getElementsByClassName("message");

console.log(messages[0]);
console.log(messages[1]);
```

This method returns a collection, so individual elements can be accessed by their indexes.

12.3 Using querySelector()

`querySelector()` selects the first element matching a CSS selector.

```
let heading = document.querySelector("h1");

let message = document.querySelector(".message");

let student = document.querySelector("#student");
```

12.4 Using querySelectorAll()

`querySelectorAll()` selects all matching elements.

```
let paragraphs = document.querySelectorAll("p");

paragraphs.forEach(function(paragraph) {

    console.log(paragraph.textContent);

});
```

12.5 Changing Text Content

HTML:

```
<h2 id="title">Old Heading</h2>
```

JavaScript:

```
document.getElementById("title").textContent =
    "New Heading";
```

12.6 Changing HTML Content

```
document.getElementById("title").innerHTML =
    "<i>Welcome to JavaScript</i>";
```

Only insert trusted or safely sanitised HTML.

12.7 Changing CSS Through JavaScript

HTML:

```
<p id="message">Welcome Students</p>
```

JavaScript:

```
let message = document.getElementById("message");

message.style.color = "green";
message.style.fontSize = "30px";
message.style.backgroundColor = "yellow";
```

JavaScript can change CSS properties through the element's `style` object.

12.8 Changing CSS Classes

HTML:

```
<p id="message">Welcome</p>
```

CSS:

```
.highlight {
    background-color: yellow;
    color: darkgreen;
    padding: 10px;
}
```

JavaScript:

```
document.getElementById("message").classList.add("highlight");
```

Other useful methods:

```
element.classList.remove("highlight");
element.classList.toggle("highlight");
element.classList.contains("highlight");
```

12.9 Changing HTML Attributes

HTML:

```
<img id="photo" src="images/old.jpg" alt="Old image">
```

JavaScript:

```
document.getElementById("photo").setAttribute(
    "src",
    "images/new.jpg"
);
```

You can retrieve an attribute using:

```
let image = document.getElementById("photo");

console.log(image.getAttribute("src"));
```

12.10 Creating HTML Elements

```
let paragraph = document.createElement("p");

paragraph.textContent = "This paragraph was created using JavaScript.";

document.body.appendChild(paragraph);
```

12.11 Removing Elements

```
let paragraph = document.getElementById("message");

paragraph.remove();
```

CHAPTER 13: JAVASCRIPT EVENTS

An event is an action that JavaScript can respond to.

13.1 Button Click Event

HTML:

```
<button id="myButton">Click Me</button>

<p id="result"></p>
```

JavaScript:

```
document.getElementById("myButton").addEventListener(
    "click",
    function() {
```

```
        document.getElementById("result").textContent =
            "Button clicked successfully!";

    }
);
```

13.2 Mouseover Event

HTML:

```
<p id="message">Move your mouse over this text.</p>
```

JavaScript:

```
let message = document.getElementById("message");

message.addEventListener("mouseover", function() {

    message.style.color = "red";

});
```

13.3 Mouseout Event

```
message.addEventListener("mouseout", function() {

    message.style.color = "black";

});
```

13.4 Double-Click Event

```
<button id="doubleBtn">Double Click Me</button>
document.getElementById("doubleBtn").addEventListener(
    "dblclick",
    function() {

        alert("You double-clicked the button!");

    }
);
```

13.5 Keyboard Events

HTML:

```
<input type="text" id="myInput" placeholder="Type something">
<p id="output"></p>
```

JavaScript:

```
document.getElementById("myInput").addEventListener(
    "keydown",
    function(event) {

        document.getElementById("output").textContent =
            "You pressed: " + event.key;

    }
);
```

The `event` object contains information about the keyboard action.

13.6 Input Event

The `input` event runs whenever the value of an input field changes.

```
let input = document.getElementById("myInput");

input.addEventListener("input", function() {

    document.getElementById("output").textContent =
        input.value;

});
```

13.7 Page Loading Event

```
document.addEventListener("DOMContentLoaded", function() {

    console.log("The HTML document has been loaded and parsed.");

});
```

This is useful when JavaScript is loaded before the end of the HTML body.

CHAPTER 14: FORM HANDLING AND VALIDATION

JavaScript can check information entered into forms before allowing submission.

14.1 Creating a Simple Form

HTML:

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Form</title>
</head>

<body>

    <h2>Student Registration</h2>

    <form id="registrationForm">

        <label>Full Name:</label>
        <input type="text" id="name">

        <br><br>

        <label>Email:</label>
        <input type="email" id="email">

        <br><br>

        <button type="submit">Register</button>

    </form>

    <p id="message"></p>

    <script src="script.js"></script>

</body>
</html>
```

JavaScript:

```
document.getElementById("registrationForm").addEventListener(
    "submit",
    function(event) {

        event.preventDefault();

        let name = document.getElementById("name").value.trim();
        let email = document.getElementById("email").value.trim();

        if (name === "" || email === "") {

            document.getElementById("message").textContent =
                "Please fill in all fields.";

        } else {

            document.getElementById("message").textContent =
                "Registration successful for " + name;

        }
```

```
    }
);
```

Explanation

- `submit` detects form submission.
- `event.preventDefault()` prevents the browser's default form submission.
- `.value` retrieves the information entered.
- `.trim()` removes unnecessary spaces from the beginning and end.
- `if` checks whether the required information is available.
- `textContent` displays the result.

Important: This example only demonstrates browser-side validation. It does not save the registration to a database. Real applications must validate submitted information on the server as well.

14.2 Checking Password Length

```
let password = "12345678";

if (password.length >= 8) {

    console.log("Password accepted");

} else {

    console.log("Password must have at least 8 characters");

}
```

14.3 Confirming Passwords

```
let password = "RuralNet123";
let confirmPassword = "RuralNet123";

if (password === confirmPassword) {

    console.log("Passwords match");

} else {

    console.log("Passwords do not match");

}
```

14.4 Checking an Email Field

HTML's `type="email"` provides basic browser validation. JavaScript can also check that a field is not empty, but a simple regular expression is not a complete guarantee that an address exists.

```
let email = document.getElementById("email").value.trim();

if (email === "") {

    console.log("Email is required");

} else {

    console.log("Email has been entered");

}
```

CHAPTER 15: JAVASCRIPT CALCULATIONS

JavaScript is useful for automating mathematical operations.

15.1 Simple Calculator

HTML:

```
<h2>Simple Calculator</h2>

<input type="number" id="num1" placeholder="First number">

<input type="number" id="num2" placeholder="Second number">

<button id="addBtn">Add</button>

<p id="answer"></p>
```

JavaScript:

```
document.getElementById("addBtn").addEventListener(
    "click",
    function() {

        let a = Number(document.getElementById("num1").value);
        let b = Number(document.getElementById("num2").value);

        let total = a + b;

        document.getElementById("answer").textContent =
            "Answer: " + total;

    }
);
```

15.2 Calculator with Four Operations

HTML:

```
<h2>Basic Calculator</h2>

<input type="number" id="a" placeholder="First number">
<input type="number" id="b" placeholder="Second number">

<br><br>

<button onclick="calculate('+')">Addition</button>
<button onclick="calculate('-')">Subtraction</button>
<button onclick="calculate('*')">Multiplication</button>
<button onclick="calculate('/')">Division</button>

<h3 id="result"></h3>
```

JavaScript:

```
function calculate(operator) {

    let a = Number(document.getElementById("a").value);
    let b = Number(document.getElementById("b").value);

    let result;

    if (operator === "+") {

        result = a + b;

    } else if (operator === "-") {

        result = a - b;

    } else if (operator === "*") {

        result = a * b;

    } else if (operator === "/") {

        if (b === 0) {

            document.getElementById("result").textContent =
                "Cannot divide by zero.";

            return;

        }

        result = a / b;

    }
```

```
    document.getElementById("result").textContent =
        "Result: " + result;

}
```

15.3 Student Fees Calculator

HTML:

```
<h2>Student Fees Calculator</h2>

<label>Total Expected Fees:</label>
<input type="number" id="totalFees">

<br><br>

<label>Amount Paid:</label>
<input type="number" id="amountPaid">

<br><br>

<button onclick="calculateFees()">Calculate</button>

<p id="balance"></p>
```

JavaScript:

```
function calculateFees() {

    let total = Number(document.getElementById("totalFees").value);
    let paid = Number(document.getElementById("amountPaid").value);

    if (
        document.getElementById("totalFees").value === "" ||
        document.getElementById("amountPaid").value === "" ||
        !Number.isFinite(total) ||
        !Number.isFinite(paid) ||
        total < 0 ||
        paid < 0
    ) {

        document.getElementById("balance").textContent =
            "Enter valid amounts.";

        return;

    }

    let balance = total - paid;

    document.getElementById("balance").textContent =
        "Outstanding Balance: KSh " + balance;

}
```

Note: This is a basic calculator. A real student fees system should also define how overpayments are handled.

CHAPTER 16: JAVASCRIPT STRINGS AND MATH METHODS

16.1 Changing Text to Uppercase

```
let name = "ruralnet training college";

console.log(name.toUpperCase());
```

16.2 Changing Text to Lowercase

```
let name = "RURALNET TRAINING COLLEGE";

console.log(name.toLowerCase());
```

16.3 Removing Extra Spaces

```
let name = "   Boaz   ";

console.log(name.trim());
```

16.4 Finding String Length

```
let message = "Welcome to JavaScript";

console.log(message.length);
```

16.5 Replacing Text

```
let message = "Welcome to HTML";

let result = message.replace("HTML", "JavaScript");

console.log(result);
```

16.6 Using Math.round()

```
console.log(Math.round(4.6));
```

Output:

```
5
```

16.7 Using Math.floor()

```
console.log(Math.floor(4.9));
```

Output:

```
4
```

16.8 Using Math.ceil()

```
console.log(Math.ceil(4.1));
```

Output:

```
5
```

16.9 Generating Random Numbers

```
let number = Math.random();

console.log(number);
```

To generate a whole number between 1 and 10:

```
let number = Math.floor(Math.random() * 10) + 1;

console.log(number);
```

CHAPTER 17: CREATING INTERACTIVE WEBPAGE FEATURES

This chapter combines HTML, CSS and JavaScript to create features commonly found on modern websites.

17.1 Show and Hide Content

HTML:

```
<h2>Important Information</h2>
```

```
<p id="info">This is some important information.</p>

<button id="hideBtn">Hide Information</button>
<button id="showBtn">Show Information</button>
```

CSS:

```
#info {
    background-color: lightyellow;
    padding: 15px;
}
```

JavaScript:

```
let info = document.getElementById("info");

document.getElementById("hideBtn").addEventListener(
    "click",
    function() {

        info.style.display = "none";

    }
);

document.getElementById("showBtn").addEventListener(
    "click",
    function() {

        info.style.display = "block";

    }
);
```

17.2 Change Background Colour

HTML:

```
<button id="changeColor">Change Background</button>
```

JavaScript:

```
document.getElementById("changeColor").addEventListener(
    "click",
    function() {

        document.body.style.backgroundColor = "lightblue";

    }
);
```

17.3 Dark Mode

HTML:

```
<button id="darkMode">Toggle Dark Mode</button>

<h1>Welcome to My Website</h1>
<p>This page supports dark mode.</p>
```

CSS:

```
body {
    background-color: white;
    color: black;
    font-family: Arial;
    transition: 0.3s;
}

body.dark {
    background-color: #222;
    color: white;
}
```

JavaScript:

```
document.getElementById("darkMode").addEventListener(
    "click",
    function() {

        document.body.classList.toggle("dark");

    }
);
```

17.4 Image Changer

HTML:

```
<img
    id="myImage"
    src="images/image1.jpg"
    width="300"
    alt="Sample image"
>

<br><br>

<button id="changeImage">Change Image</button>
```

JavaScript:

```
let image = document.getElementById("myImage");

document.getElementById("changeImage").addEventListener(
```

```
    "click",
    function() {

        image.src = "images/image2.jpg";

    }
);
```

Place both images in the `images` folder.

17.5 Character Counter

HTML:

```
<textarea id="message" maxlength="200"></textarea>

<p id="counter">0 / 200 characters</p>
```

JavaScript:

```
let message = document.getElementById("message");

message.addEventListener("input", function() {

    let count = message.value.length;

    document.getElementById("counter").textContent =
        count + " / 200 characters";

});
```

17.6 Floating Modal Box

A modal is a popup window displayed over a webpage.

HTML:

```
<button id="openModal">Open Information</button>

<div id="overlay" class="overlay">

    <div class="modal">

        <button id="closeModal" class="close">×</button>

        <h2>Welcome!</h2>

        <p>This is a floating information box.</p>

    </div>

</div>
```

CSS:

```
.overlay {
    display: none;
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.4);
    backdrop-filter: blur(5px);
    justify-content: center;
    align-items: center;
}

.modal {
    background: white;
    width: 350px;
    max-width: 90%;
    padding: 25px;
    border-radius: 12px;
    position: relative;
    box-shadow: 0 5px 20px rgba(0,0,0,0.3);
}

.close {
    position: absolute;
    right: 10px;
    top: 10px;
    border: none;
    background: none;
    font-size: 25px;
    cursor: pointer;
}
```

JavaScript:

```
let overlay = document.getElementById("overlay");

document.getElementById("openModal").addEventListener(
    "click",
    function() {

        overlay.style.display = "flex";

    }
);

document.getElementById("closeModal").addEventListener(
    "click",
    function() {

        overlay.style.display = "none";

    }
);

overlay.addEventListener("click", function(event) {
```

```
    if (event.target === overlay) {
        overlay.style.display = "none";
    }
});
```

This exercise demonstrates event handling, DOM manipulation, CSS positioning and modal visibility.

CHAPTER 18: BROWSER STORAGE USING LOCALSTORAGE

`localStorage` allows JavaScript to save information in the user's browser.

18.1 Saving Data

```
localStorage.setItem("username", "Boaz");
```

18.2 Retrieving Data

```
let name = localStorage.getItem("username");
console.log(name);
```

18.3 Removing One Item

```
localStorage.removeItem("username");
```

18.4 Clearing All Items

```
localStorage.clear();
```

18.5 Saving User Input

HTML:

```
<input type="text" id="name" placeholder="Enter your name">
<button id="saveName">Save Name</button>
<p id="displayName"></p>
```

JavaScript:

```
document.getElementById("saveName").addEventListener(
    "click",
    function() {

        let name = document.getElementById("name").value.trim();

        if (name === "") {

            alert("Please enter your name.");
            return;

        }

        localStorage.setItem("studentName", name);

        document.getElementById("displayName").textContent =
            "Saved name: " + name;

    }
);
```

18.6 Displaying Previously Saved Data

```
let savedName = localStorage.getItem("studentName");

if (savedName !== null) {

    document.getElementById("displayName").textContent =
        "Welcome back, " + savedName;

}
```

Important: Browser storage is suitable for simple preferences and demonstrations. It should not be treated as a secure database or used to store passwords and secret credentials.

CHAPTER 19: WORKING WITH JSON

JSON is a common format for storing and exchanging data.

19.1 Creating a JavaScript Object

```
let student = {

    name: "Mary",
    course: "ICT Essentials",
    fees: 25000
```

```
};
```

19.2 Converting an Object to JSON

```
let jsonData = JSON.stringify(student);

console.log(jsonData);
```

19.3 Converting JSON to a JavaScript Object

```
let originalData = JSON.parse(jsonData);

console.log(originalData.name);
```

19.4 Saving an Object in localStorage

```
let student = {

    name: "Mary",
    course: "ICT Essentials",
    fees: 25000

};

localStorage.setItem("student", JSON.stringify(student));
```

Retrieving the object:

```
let savedStudent = JSON.parse(
    localStorage.getItem("student")
);

if (savedStudent !== null) {

    console.log(savedStudent.name);

}
```

CHAPTER 20: INTRODUCTION TO FETCH API

The Fetch API allows JavaScript to retrieve information from a server.

20.1 Basic Fetch Example

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(function(response) {

        if (!response.ok) {
            throw new Error("Network response was not successful");
        }

        return response.json();

    })
    .then(function(data) {

        console.log(data);

    })
    .catch(function(error) {

        console.error("Error:", error);

    });
```

This example retrieves sample user information from a public demonstration API.

20.2 Using async and await

```
async function getUsers() {

    try {

        let response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
            throw new Error("Failed to retrieve users");
        }

        let data = await response.json();

        console.log(data);

    } catch (error) {

        console.error(error);

    }

}

getUsers();
```

20.3 Displaying Retrieved Information on a Webpage

HTML:

```
<button id="loadUsers">Load Users</button>

<ul id="users"></ul>
```

JavaScript:

```
document.getElementById("loadUsers").addEventListener(
    "click",
    async function() {

        let list = document.getElementById("users");

        list.textContent = "Loading...";

        try {

            let response = await fetch(
                "https://jsonplaceholder.typicode.com/users"
            );

            if (!response.ok) {
                throw new Error("Unable to retrieve users");
            }

            let users = await response.json();

            list.replaceChildren();

            users.forEach(function(user) {

                let item = document.createElement("li");

                item.textContent = user.name;

                list.appendChild(item);

            });

        } catch (error) {

            list.textContent = "An error occurred while loading users.";

            console.error(error);

        }

    }
);
```

Note: Fetch requires a working network connection for this public API example. Real applications also need to consider API permissions, authentication, server validation and error handling.

CHAPTER 21: JAVASCRIPT ERROR HANDLING AND DEBUGGING

21.1 Using console.log()

```
let total = 100 + 200;

console.log(total);
```

21.2 Using console.error()

```
console.error("An error occurred!");
```

21.3 Using try and catch

```
try {
    let result = 10 / 0;
    console.log(result);
} catch (error) {
    console.log("An error occurred.");
}
```

JavaScript does not automatically throw an exception for ordinary division by zero. The example illustrates the structure of `try...catch`, not a division-by-zero error.

21.4 Throwing an Error

```
function checkAge(age) {
    if (age < 0) {
        throw new Error("Age cannot be negative.");
    }
    return age;
}
try {
```

```
    console.log(checkAge(-5));

} catch (error) {

    console.log(error.message);

}
```

21.5 Debugging with Browser Developer Tools

1. Open your webpage.
2. Press F12.
3. Select Console.
4. Look for red error messages.
5. Click an error to locate the relevant code.
6. Correct the error in VS Code.
7. Save and reload the webpage.

Common beginner errors:

- Misspelling variable names.
- Forgetting quotation marks.
- Using `=` instead of `===` in a comparison.
- Selecting an incorrect HTML ID.
- Forgetting to connect the JavaScript file.
- Placing a script before required HTML elements without using `defer` or waiting for the DOM.
- Forgetting to convert input values into numbers.

CHAPTER 22: FINAL INTEGRATED PRACTICAL PROJECT

PROJECT: STUDENT REGISTRATION AND FEES MANAGEMENT INTERFACE

Develop a small webpage that allows a user to enter student information and calculate the student's fee balance.

Requirements

Part A: HTML

Create a form containing:

1. Full name.
2. Registration number.
3. Course.
4. Total expected fees.
5. Amount paid.
6. Telephone number.

Part B: CSS

Style the webpage using:

1. A coloured background.
2. A centred form.
3. Rounded corners.
4. Box shadows.
5. Properly styled input fields.
6. A responsive layout.
7. A clearly visible submit button.

Part C: JavaScript

Your program should:

1. Retrieve information entered into the form.
2. Check that required fields are completed.
3. Check that fee amounts are valid.
4. Calculate the outstanding balance.
5. Calculate the percentage of fees paid.
6. Display the student details.
7. Display the calculated balance.
8. Display a suitable success or error message.

Part D: Additional Challenge

Extend the application to:

1. Add multiple students to an array.
2. Display students in an HTML table.
3. Search for a student using their registration number.
4. Delete a student record from the displayed list.
5. Save the records in `localStorage`.
6. Retrieve saved records when the page is reopened.

Important: This project uses browser storage for learning purposes. It is not a replacement for a secure database-backed student management system.

GENERAL JAVASCRIPT PRACTICAL ASSIGNMENTS

Assignment 1: Personal Information Display

Create a webpage that asks the user for their name, age, gender and course, then displays the information.

Assignment 2: Simple Calculator

Create a calculator that performs addition, subtraction, multiplication and division.

Assignment 3: Student Grading System

Accept examination marks and display the corresponding grade.

Assignment 4: Multiplication Table

Ask the user to enter a number and display its multiplication table from 1 to 12.

Assignment 5: Even and Odd Checker

Accept a number and determine whether it is even or odd.

Assignment 6: Password Checker

Create a password field and a button that allows users to show or hide the entered password.

Assignment 7: Character Counter

Create a textarea that counts the number of characters entered by a user.

Assignment 8: Image Gallery

Create a webpage with several images. Allow the user to click a button to move between images.

Assignment 9: Student Registration Form

Create a registration form with JavaScript validation.

Assignment 10: To-Do List

Create a webpage that allows users to:

1. Add tasks.
2. Display tasks.
3. Mark tasks as completed.
4. Delete tasks.
5. Save tasks using localStorage.

EXPECTED PRACTICAL LEARNING OUTCOMES

By the end of this practical section, students should be able to:

1. Connect JavaScript files to HTML documents.
2. Display output using JavaScript.
3. Declare variables and manipulate data.
4. Use operators to perform calculations.
5. Receive and process user input.
6. Use conditional statements and loops.
7. Create and call functions.
8. Work with arrays and objects.
9. Select and manipulate HTML elements through the DOM.
10. Respond to user actions using events.
11. Validate HTML forms.
12. Change CSS properties and classes using JavaScript.
13. Create interactive webpage features.
14. Store and retrieve simple browser data.
15. Work with JSON and introductory APIs.
16. Identify and correct common JavaScript errors.
17. Combine HTML, CSS and JavaScript to develop functional webpages.

RECOMMENDED TEACHING ORDER

Teach the practical section in the following sequence:

1. Project setup and connecting JavaScript to HTML.
2. Output methods and variables.
3. Data types, operators and input.
4. Conditions and loops.
5. Functions.
6. Arrays and objects.
7. DOM selection and manipulation.
8. Events.
9. Form handling and validation.
10. Calculations.
11. Interactive webpage features.
12. Browser storage.
13. JSON and Fetch API.
14. Debugging.
15. Integrated student management project.

Teaching advice: Students should type, run, test and modify every example themselves. Avoid simply giving them completed code to copy. After demonstrating an example, change some values or requirements and ask students to modify the program independently.